

借贷协议期末考试

进阶版 · SimpleLendingPool + Compound V2 原理

课程：智能合约安全审计	总分：100 分	建议时长：60 分钟
题型：单选 5 题 + 计算 3 题 + 分析 2 题	答案：见附录（第 3 页）	允许：草稿纸 / 代码参考

注意事项

1. 选择题每题只有一个正确答案；选项描述均在技术上成立，请仔细区分。
2. 计算题需写出完整推导过程，最终结果保留合理精度。
3. 分析题请结合合约代码作答，泛泛而谈不得分。

第一部分 单选题（每题 6 分，共 30 分）

1

(6
分)

► 陷阱

协议运行一段时间后，Alice 决定追加存款。此时 $\text{exchangeRate} > 1$ 。与协议刚启动时相比，Alice 追加相同数量的 `borrowAsset`，会获得：

- A 更多 IToken，因为池子变大了，每份 IToken 代表更多资产
- B 更少 IToken，因为 exchangeRate 上升意味着每份 IToken 更值钱
- C 相同数量 IToken， exchangeRate 只影响取款，不影响存款
- D 取决于当前利用率，利用率高则 IToken 多，低则少

2

(6
分)

► 陷阱

Bob 第一次在 $\text{borrowIndex} = 1.0 \times \text{RAY}$ 时借了 500 USDC，之后 borrowIndex 涨到 $1.05 \times \text{RAY}$ 时他又追加借了 100 USDC。追加借款后，Bob 的 `BorrowSnapshot.principal` (`normalizedPrincipal`) 应该是多少？

- A $500 \times \text{RAY} + 100 \times \text{RAY} = 600 \times \text{RAY}$ （两次直接累加）
- B $500 \times \text{RAY} + 100 / 1.05 \times \text{RAY} \approx 595.24 \times \text{RAY}$ （旧 principal 加新标准化额）
- C 先将旧债务按新 index 折算后再合并： $(500 \times 1.05 + 100) / 1.05 \times \text{RAY} \approx 595.24 \times \text{RAY}$
- D $100 / 1.05 \times \text{RAY} \approx 95.24 \times \text{RAY}$ （只记录追加部分，旧快照保留）

3

(6分)

► 陷阱

资金池当前没有任何借款 ($\text{totalBorrows} = 0$)，池中有 10000 USDC 存款。此时若有人尝试存款，其 IToken 的 mint 数量计算中 exchangeRate 等于多少？

- A 0，因为 $\text{totalBorrows} = 0$ ，分子为零
- B $1e18$ (WAD)，合约对 $\text{ITokenSupply} = 0$ 做了特殊处理，直接返回 WAD
- C $10000e18 / \text{IToken.totalSupply}()$ ，与 totalBorrows 无关
- D 无法确定，取决于上次 $\text{accrueInterest}()$ 调用时的 borrowIndex

4

(6分)

► 陷阱

某借款人健康因子为 0.72，清算人代还了其 50% 的债务并取走了相应抵押物 (含 5% bonus)。清算完成后，该借款人的健康因子最可能是：

- A 恰好等于 1.0，协议保证单次清算将健康因子精确修复到 1
- B 仍然小于 1，因为原始健康因子过低，单次 50% 清算不足以修复
- C 大于 1，因为还了一半债务，剩余抵押物比例上升
- D 无法确定，取决于 LIQUIDATION_BONUS 和价格的乘积关系

5

(6分)

► 推导

下面哪条语句最准确地描述了 SimpleLendingPool 中 $\text{getTotalBorrows}()$ 的行为？

- A 总是返回最新债务，因为 Solidity view 函数在调用时自动刷新状态
- B 返回上次 $\text{accrueInterest}()$ 执行后冻结的债务快照，不含此后的利息
- C 每次调用都会触发一次内部 $\text{accrueInterest}()$ ，保证数据实时
- D 返回 $\text{totalBorrowPrincipal}$ ，即未含利息的标准化本金之和

第二部分 计算题（每题 14 分，共 42 分）

6

(14
分)

► 综合

资金池参数: $\text{BASE_RATE}=2\%$, $\text{SLOPE1}=10\%$, $\text{SLOPE2}=500\%$, $\text{OPTIMAL}=80\%$, $\text{SECONDS_PER_YEAR}=365$ 天。

当前状态: 池中剩余 200 USDC, 已借出 800 USDC, $\text{borrowIndex} = 1.0 \times \text{RAY}$, 距上次 accrue 恰好 30 天。

- ① 计算当前利用率 (精确到小数点后 1 位)
- ② 计算当前年化借款利率 APR (精确到小数点后 2 位)
- ③ 计算 30 天后执行 $\text{accrueInterest}()$ 时新的 borrowIndex (以 RAY 为单位, 保留 6 位有效数字)
- ④ 若 Carol 在此时 (30 天后) 借了 100 USDC, 写出她的 $\text{BorrowSnapshot.principal}$

① 利用率 =

② $\text{APR} =$

③ 新 $\text{borrowIndex} =$

④ $\text{principal} =$

7

(14
分)

► 推导

Dave 的借款历史:

- 时刻 T1: $\text{borrowIndex}=1.0 \times \text{RAY}$, 借了 1000 USDC
- 时刻 T2: $\text{borrowIndex}=1.10 \times \text{RAY}$, 追加借了 500 USDC
- 时刻 T3: $\text{borrowIndex}=1.25 \times \text{RAY}$, Dave 全额还清

- ① T2 追加借款后, Dave 的 $\text{BorrowSnapshot.principal}$ 是多少?
- ② T3 时 Dave 需要还多少 USDC (实际总债务)?
- ③ Dave 这笔借款总共支付了多少利息?

① T2 后 $\text{principal} =$

② T3 还款金额 =

③ 总利息 =

8

(14
分)

► 推导

Eve 存入了 X WETH 作为抵押物，借出了恰好等于 LTV 上限的 USDC。

合约参数：COLLATERAL_PRICE=2，LTV=75%，

LIQUIDATION_THRESHOLD=80%，LIQUIDATION_BONUS=5%。

- ① 写出 Eve 初始健康因子的表达式（用 LTV 和 LIQUIDATION_THRESHOLD 表示）
- ② Eve 被清算时，清算人代还了全部债务。写出清算人获得的抵押物数量（用 X 表示）
- ③ 清算完成后，Eve 剩余多少 WETH？当 X=100 时，具体数字是多少？
- ④ 若清算后 Eve 账户出现「抵押物不足以支付 bonus」的情况，合约会如何处理？

① 初始 HF =

② 清算人获得 WETH =

③ Eve 剩余 WETH =

④ 合约处理：

第三部分 代码分析题（每题 14 分，共 28 分）

9

(14
分)

► 代码

下面是一个攻击者尝试对 SimpleLendingPool 进行「多次存款套利」的场景：

```
// 攻击者在同一个 block 内连续操作
pool.deposit(10_000e18);           // 操作 A
pool.borrow(7_500e18);             // 操作 B（假设已存入足量抵押物）
// ... 等待 1 个 block
pool accrueInterest();              // 操作 C
pool.repay(pool.getBorrowBalance(address(this))); // 操作 D
```

```
pool.withdraw(lToken.balanceOf(address(this))); // 操作 E
```

问题:

- ① 操作 A 和 B 在同一个 block 内发生, 此时 `accrueInterest()` 会做什么? 为什么?
- ② 操作 C 之后, 借款人的债务比操作 B 时增加了多少? (设 1 个 block = 12s, APR 按计算题 Q6 的结果代入)
- ③ 上述操作序列中是否存在可供攻击者获利的漏洞? 请说明理由。若存在, 修复思路是什么?

10

(14
分)

► 分析

审计 `repay()` 函数时, 你发现以下代码段:

```
function repay(uint256 amount) external {
    accrueInterest();
    uint256 debt = _getBorrowBalance(msg.sender);
    uint256 repayAmount = amount > debt ? debt : amount;

    // ← 以下是计算「本次还款中有多少是利息」
    uint256 interest = repayAmount > (borrows[msg.sender].principal *
borrowIndex / RAY)
        ? repayAmount - (borrows[msg.sender].principal * borrowIndex /
RAY)
        : 0;

    borrowAsset.transferFrom(msg.sender, address(this), repayAmount);
    uint256 newDebt = debt - repayAmount;
    uint256 newPrincipal = (newDebt * RAY) / borrowIndex;
    totalBorrowPrincipal = totalBorrowPrincipal
        - borrows[msg.sender].principal + newPrincipal;
    borrows[msg.sender] = BorrowSnapshot({ principal: newPrincipal,
        interestIndex: borrowIndex });
}
```

- ① 指出 `interest` 变量的计算存在的精度问题，并说明在什么情况下它会出错。
- ② $\text{newPrincipal} = (\text{newDebt} * \text{RAY}) / \text{borrowIndex}$ 存在向下取整。
设 $\text{newDebt} = 1$ (wei)， $\text{borrowIndex} = 1.5 \times \text{RAY}$ ，计算 `newPrincipal` 的值，
并分析这会对协议产生什么持续性影响（每次还 1 wei 反复操作）。
- ③ `totalBorrowPrincipal` 的更新是否可能导致下溢？在什么前提下安全？

附录 · 参考答案

答案仅供参考，分析类题目言之有理均可酌情给分

第一部分 第一部分 · 单选题答案

题 1 存款时 IToken 铸造数量

正确答案：B

干扰分析：

A 听起来合理（池子大了=每份代表更多），但逻辑方向相反。

`exchangeRate` 上升 → 相同 `borrowAsset` 能换到的 IToken 更少（分母变大）。

C 错误：`exchangeRate` 在存款和取款时都使用，双向影响。

D 错误：IToken 数量 = `deposit / exchangeRate`，与利用率无直接关联。

关键公式：`mintAmount = depositAmount / exchangeRate`

`exchangeRate`↑ → `mintAmount`↓，存款人收益体现在取款时兑换更多，而非持有更多 IToken。

题 2 追加借款时 `normalizedPrincipal` 的合并

正确答案：C

干扰分析：

A 错：直接累加两次 `principal` 忽略了 `index` 的差异，会高估旧债利息。

B 错：旧 `principal(500×RAY)` 是在 `index=1.0` 时登记的，直接加新标准化额会造成双重计息（旧 `principal` 已经隐含了 0→1.05 的利息增长）。

D 错：若只记追加部分，则旧债务的 `index` 快照就丢失了。

正确逻辑：先用当前 `index` 还原旧债实际金额，加上新借款，再整体重新标准化：

`existingDebt = 500×RAY × 1.05×RAY / RAY = 525 USDC`

`newDebt = 525 + 100 = 625 USDC`

`newPrincipal = 625 / 1.05 × RAY ≈ 595.238×RAY`

这与 C 选项描述的操作一致，也是合约代码中 `borrow()` 的实际做法。

题 3 ITokenSupply=0 时的 `exchangeRate`

正确答案：B

干扰分析：

A 错：分子不是 `totalBorrows`，而是 `totalAssets = cash + totalBorrows`。即使 `totalBorrows=0`，`cash=10000e18` 也不为零；但更关键的是分母 `lTokenSupply=0` 会导致除以零。

C 看起来正确（公式正确），但当 `supply=0` 时合约做了特殊处理（提前 `return WAD`），所以实际执行路径不同，`10000e18 / 0` 不会被执行。

D 错：`exchangeRate` 与 `borrowIndex` 无直接依赖关系。

合约代码逻辑：`if (supply == 0) return WAD; // 初始 1:1`

因此第一个存款人始终按 1:1 获得 IToken，这保证了协议可以从零启动。

题 4 清算后健康因子

正确答案：D

干扰分析（这题最难，四个选项在不同条件下都可能成立）：

A 错：协议没有任何机制保证单次清算精确修复到 1.0。

B 部分正确：当初始 HF 极低时（如 0.3）确实可能仍 < 1 ，但不是必然。

C 部分正确：当债务减少幅度大于 bonus 消耗时，HF 确实会上升超过 1。

严格分析：设清算前 $\text{debt}=D$, $\text{collateral}=C$, $\text{threshold}=T$, $\text{bonus}=b$, $\text{price}=p$

清算后: $\text{debt}' = D/2$, $\text{collateral}' = C - (D/2)/p \cdot (1+b)$

$\text{HF}' = \text{collateral}' \cdot p \cdot T / \text{debt}'$

当 $\text{HF_initial}=0.72$ 时可能恢复到 >1 ，也可能仍 <1 ，取决于具体数值。

所以 D（无法确定）才是最严谨的答案。

题 5 getTotalBorrows() 的行为

正确答案：B

干扰分析：

A 错：Solidity view 函数不会触发任何状态更新，这是基本语言特性。

C 错：getTotalBorrows() 是纯计算函数，不会调用 accrueInterest()（后者修改状态）。

D 错：getTotalBorrows() = totalBorrowPrincipal × borrowIndex / RAY，
这包含了 borrowIndex，不是纯 principal。

关键：borrowIndex 是状态变量，只在 accrueInterest() 被调用时更新。

getTotalBorrows() 使用的是当前存储的 borrowIndex（可能是过时的），
所以返回的是「上次 accrue 之后冻结的债务状态」。

第二部分 第二部分 · 计算题答案

题 6 多步利率计算 + borrowIndex 更新

① 利用率：

$$U = \text{totalBorrows} / (\text{cash} + \text{totalBorrows}) = 800 / (200 + 800) = 80.0\%$$

② APR（U = 80% 恰好在 kink 点，用低区间公式）：

$$\begin{aligned} \text{rate} &= \text{BASE_RATE} + \text{SLOPE1} \times (U / \text{OPTIMAL}) \\ &= 2\% + 10\% \times (80\% / 80\%) = 2\% + 10\% = 12.0\% \end{aligned}$$

注：若用高区间公式（U > OPTIMAL）得到相同结果，因为 80% 是边界点。

③ 新 borrowIndex（30 天，线性近似）：

$$\begin{aligned} \text{elapsed} &= 30 \times 86400 = 2,592,000 \text{ 秒} \\ \text{interestFactor} &= \text{annualRate} \times \text{elapsed} / \text{SECONDS_PER_YEAR} \\ &= 0.12 \times 2,592,000 / 31,536,000 = 0.12 \times 0.082192... \approx \\ &0.0098630 \\ \text{newIndex} &= \text{RAY} \times (1 + 0.0098630) \approx 1.009863 \times \text{RAY} \end{aligned}$$

即 $\text{borrowIndex} \approx 1.00986 \times \text{RAY}$ (6 位有效数字)

④ Carol 的 `BorrowSnapshot.principal`:

$\text{principal} = 100 \text{ USDC} / \text{borrowIndex} \times \text{RAY}$
 $= 100 / 1.00986 \times \text{RAY} \approx 99.0237 \times \text{RAY}$ (即约 99.0237e27)

题 7 两次借款合并 + 全额还款

① T2 追加借款后的 `principal`:

T1 时 $\text{principal}_1 = 1000 / 1.0 \times \text{RAY} = 1000 \times \text{RAY}$
T2 时旧债实际金额 $= 1000 \times \text{RAY} \times 1.10 \times \text{RAY} / \text{RAY} = 1100 \text{ USDC}$
新总债务 $= 1100 + 500 = 1600 \text{ USDC}$
T2 后 $\text{principal} = 1600 / 1.10 \times \text{RAY} \approx 1454.545 \times \text{RAY}$

② T3 全额还款金额:

$\text{actualDebt} = \text{principal} \times \text{borrowIndex}_{\text{T3}} / \text{RAY}$
 $= 1454.545 \times \text{RAY} \times 1.25 \times \text{RAY} / \text{RAY} = 1454.545 \times 1.25$
 $= 1818.18 \text{ USDC}$ (约)

③ 总利息:

总利息 $= \text{还款金额} - \text{实际借出金额} = 1818.18 - (1000 + 500) = 318.18 \text{ USDC}$
验证: T1→T2 利息 $= 1000 \times (1.10 - 1.0) = 100$, T2→T3 利息 $= 1600 \times (1.25 - 1.10) / 1.10 \approx 218.18$
合计 $\approx 318.18 \checkmark$

题 8 清算边界与风险参数推导

① 初始健康因子 (Eve 借到 LTV 上限):

$\text{maxBorrow} = X \times \text{PRICE} \times \text{LTV} = X \times 2 \times 0.75 = 1.5X \text{ USDC}$
 $\text{HF} = (X \times \text{PRICE} \times \text{LIQUIDATION_THRESHOLD}) / \text{maxBorrow}$
 $= (X \times 2 \times 0.80) / (X \times 2 \times 0.75) = 0.80 / 0.75 = 16/15 \approx 1.0667$
即: $\text{HF} = \text{LIQUIDATION_THRESHOLD} / \text{LTV}$ (与 X 无关)

② 清算人代还全部债务 $D=1.5X$, 获得 WETH:

$\text{collateralValue} = D / \text{PRICE} = 1.5X / 2 = 0.75X \text{ WETH}$ (等值部分)
 $\text{bonus} = 0.75X \times 5\% = 0.0375X \text{ WETH}$
获得总量 $= 0.75X + 0.0375X = 0.7875X \text{ WETH}$

③ Eve 剩余 WETH:

Eve 剩余 $= X - 0.7875X = 0.2125X \text{ WETH}$
当 $X=100$ 时: Eve 剩余 $= 21.25 \text{ WETH}$

④ 抵押物不足支付 `bonus` 时合约处理:

`require(collateralBalance[borrower] >= collateralSeized)` 会 `revert`,
即合约直接回滚整笔清算交易。
风险: 若抵押物价值严重不足, 可能无人愿意清算, 协议产生坏账 (bad debt)。
生产协议 (如 Aave) 的处理: 引入 bad debt accounting 机制, 由协议储备金吸收亏损。

第三部分 第三部分·代码分析题答案

题9 同块操作与利息累积

① 操作 A、B 在同一 block 内：

`accrueInterest()` 首先检查 `elapsed = block.timestamp - lastAccrualTimestamp`。
同一 block 内 `block.timestamp` 不变, `elapsed = 0`, 函数直接 `return`, 不更新 `index`。
因此 A→B 之间无利息产生, `borrowIndex` 不变。

② 1 个 block (12s) 后债务增量：

由 Q6 结果: `APR = 12%`, `SECONDS_PER_YEAR = 31,536,000s`
`interestFactor = 0.12 × 12 / 31,536,000 ≈ 4.566e-8`
债务增量 = `7500 × 4.566e-8 ≈ 0.000342 USDC` (极小, 约 342 wei)
实际上 12 秒内利息几乎可以忽略不计。

③ 是否存在可利用漏洞：

不存在明显套利漏洞。分析：

- 存款→借款→还款→取款是正常操作序列, 不违反 CEI 模式。
 - 攻击者作为存款人 (Alice 角色) 获得利息; 作为借款人支付利息。
 - 在同一协议内存借并持有, 利息净收益接近零 (存款 $APY \approx$ 借款 $APR \times U$)。
 - 若攻击者试图通过闪电贷操纵 `exchangeRate`, 本合约无 `flashloan` 接口, 无此风险。
- 潜在风险点 (非本题重点): 若 `borrowAsset` 是 ERC777, 操作 E 的 `transfer` 可重入, 但这是独立的 `reentrancy` 问题, 与本操作序列的套利逻辑无关。

题10 精度截断漏洞分析

① `interest` 变量的精度问题：

`borrows[msg.sender].principal * borrowIndex / RAY` 存在整数截断。
等价写法: `originalDebt ≈ principal × index / RAY` (向下取整)
若 `repayAmount` 恰好等于真实 `debt`, 则 `interest` 可能被高估 1 wei (因分母截断使分母偏小)。
更严重的情况: 若 `borrowIndex` 非整数倍, 中间乘法结果被截断,
`interest` 计算结果与实际利息可能差 1~2 wei, 但不影响实际还款金额 (`repayAmount` 已确定)。
结论: `interest` 变量仅用于 `emit`, 不影响核心状态, 精度误差可接受但应注意。

② `newPrincipal` 向下取整的持续性影响：

`newDebt = 1 wei`, `borrowIndex = 1.5×RAY`
`newPrincipal = (1 × RAY) / (1.5×RAY) = RAY / (1.5×RAY) = 0` (整数除法!)
结果: 还了 1 wei 后 `newPrincipal = 0`, 系统认为债务为零, 但实际上还欠 `debt-1 wei`。
`totalBorrowPrincipal` 被错误地减少了 `borrows[msg.sender].principal` (非零值),
导致全局 `totalBorrowPrincipal` 比实际低, 进而使 `getTotalBorrows()` 和利用率被低估。
反复操作 (每次还 1 wei) 可以无代价清空债务, 同时操纵 `totalBorrowPrincipal`。
修复: 在 `repay` 入口要求 `amount >=` 某个最小值, 或对 `newDebt > 0` 但 `newPrincipal = 0` 的情况特殊处理。

③ `totalBorrowPrincipal` 下溢分析：

更新: `totalBorrowPrincipal = totalBorrowPrincipal - old + newPrincipal`
若 `newPrincipal < old` (正常还款), 结果为减小, 无溢出。

若追加借款导致 `old` 被替换为更大的 `new`，结果为增大，无溢出。
潜在下溢：若 `old > totalBorrowPrincipal`（极端情况：外部篡改或舍入积累），
Solidity 0.8+ 会 `panic revert`，保证安全性（但会锁定账户）。
前提：所有 `principal` 修改都通过 `borrow/repay/liquidate` 经过 `accrueInterest()` 后进行，则安全。

— END —